FORCE N
sa ëllëg ci xarala

Mentorat CYBERSÉCURITÉ CYB6 / Groupe 3

Encadrant : M. Ibrahima Samba DIOP
Consultant en Cybersécurité

Rapport de Lab

Attaques par déni de service (DoS) et Man-in-the-Middle (MITM)

Étudiant(e) : HAMADY CISSE

Date : 12 octobre 2026

Environnement : VirtualBox, réseau host-only `192.168.56.0/24`

Document réalisé dans un cadre strictement pédagogique et sur un laboratoire isolé.

Table des matières

1 Introduction **2**
1.1 Objectifs . . . 2

2 Environnement du laboratoire **2**
2.1 Vérification des interfaces . . . 2
2.2 Test de connectivité . . . 3

3 Tâche 1 : Attaque par déni de service avec Scapy **4**
3.1 Rappels théoriques . . . 4
3.2 Le module Scapy . . . 4
3.3 Manipulations . . . 5
3.4 Questions de la tâche 1 . . . 6

4 Tâche 2 : Analyse du trafic avec Wireshark **6**

5 Tâche 3 : Attaque DoS par SYN Flood **7**
5.1 3.1 SYN Flood avec Metasploit . . . 7
5.2 3.2 SYN Flood avec hping3 . . . 8
5.3 3.3 Démonstration de l'impact (et des défenses) . . . 9
5.3.1 État de référence (avant attaque) . . . 9
5.3.2 Tentative 1 : flood non usurpé . . . 9
5.3.3 Tentative 2 : usurpation aléatoire (--rand-source) . . . 10
5.3.4 Tentative 3 : usurpation dans le sous-réseau (réussie) . . . 10
5.4 Questions de la tâche 3 . . . 11

6 Tâche 4 : Sniffing avec Wireshark (identifiants DVWA) **11**
6.1 Questions de la tâche 4 . . . 13

7 Tâche 5 : Attaque MITM par ARP Spoofing (Ettercap) **13**
7.1 Le protocole ARP . . . 13
7.2 Scénario . . . 14
7.3 Mise en œuvre . . . 14
7.4 Questions de la tâche 5 . . . 17

8 Réflexion globale et conclusion **17**

9 Annexe : commandes utilisées, rôles et options **18**

1. Introduction

La sécurité des systèmes informatiques repose en grande partie sur la capacité à identifier, exploiter et corriger les vulnérabilités. Ce laboratoire a pour objectif de mettre en pratique plusieurs techniques de cybersécurité offensive, puis d'en tirer les contre-mesures défensives correspondantes.

Au cours de ce lab, nous explorons les attaques par déni de service (DoS), le sniffing (capture de trafic), les attaques de type Man-in-the-Middle (MITM) par empoisonnement ARP, ainsi que l'analyse du trafic avec Wireshark. Tous les exercices sont réalisés dans un cadre pédagogique strict et contrôlé.

Avertissement légal et éthique

Toutes les manipulations décrites dans ce rapport ont été réalisées exclusivement sur un laboratoire **isolé** (réseau VirtualBox host-only, sans accès Internet), composé de machines virtuelles volontairement vulnérables appartenant à l'auteur. N'effectuez jamais d'attaque (scan, DoS, MITM) sur un site, un système ou un réseau sans l'autorisation préalable et écrite du propriétaire.

1.1 Objectifs

— Comprendre les attaques par Déni de Service (DoS) afin d'analyser leurs mécanismes et impacts.
— Découvrir et capturer le trafic réseau en utilisant Wireshark.
— Réaliser des attaques MITM avec Ettercap pour intercepter les communications.
— Explorer l'exploitation de vulnérabilités et les outils associés (Metasploit, hping3, Scapy).
— Étudier les techniques de défense pour contrer les attaques DoS/DDoS, MITM et autres intrusions.

2. Environnement du laboratoire

Le laboratoire est constitué de trois machines virtuelles connectées au même réseau isolé host-only `192.168.56.0/24`.

Machine	Rôle	Adresse IP	Adresse MAC
Kali Linux	Attaquant	`192.168.56.10`	`08:00:27:aa:ac:ee`
Metasploitable 2	Cible / serveur (DVWA)	`192.168.56.30`	`08:00:27:35:23:af`
Windows 7	Victime	`192.168.56.20`	`08:00:27:d5:11:03`

Adresse de diffusion (broadcast) du réseau : `192.168.56.255`.

2.1 Vérification des interfaces

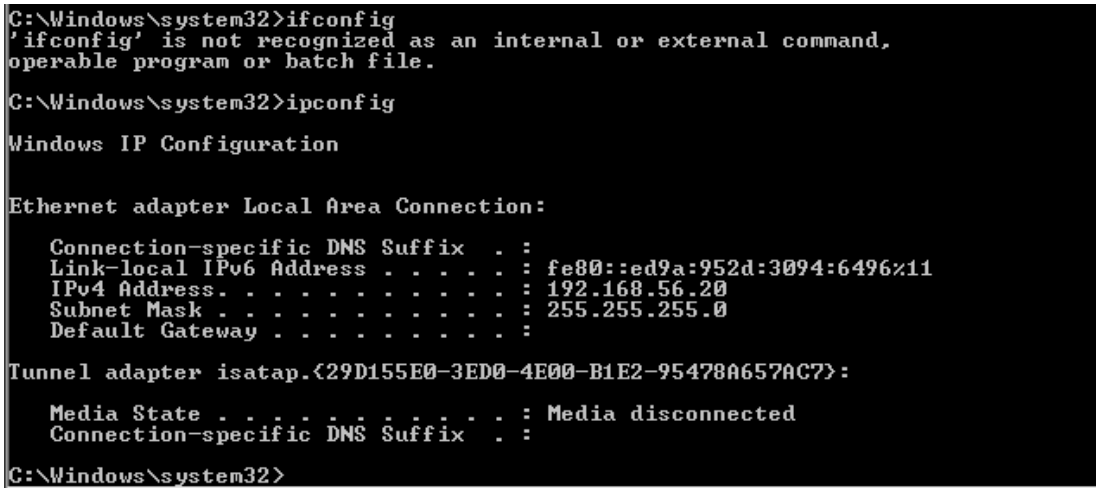
```
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:aa:ac:ee brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 scope global eth0
       valid_lft forever preferred_lft forever

┌──(kali㉿kali)-[~]
└─$
```

Kali, sortie de `ip a` *(eth0 : 192.168.56.10).*



Metasploitable 2, sortie de `ifconfig` (eth0 : 192.168.56.30).



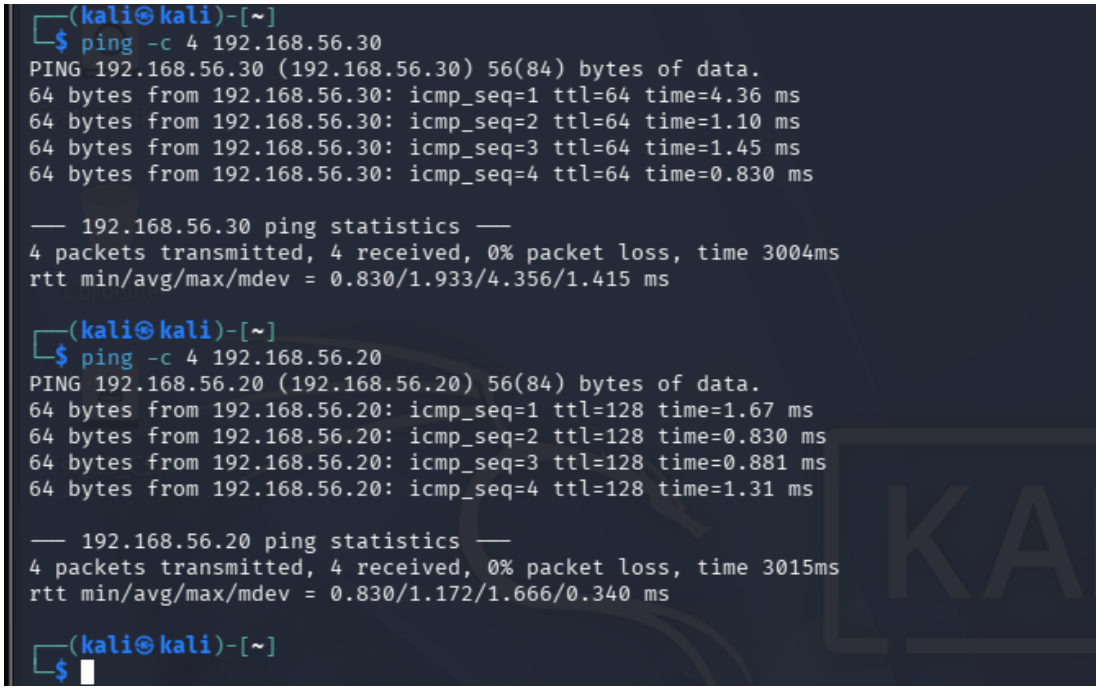
Windows 7, sortie de `ipconfig` (192.168.56.20).

2.2 Test de connectivité

Depuis Kali, on vérifie que les cibles répondent :

```
# Vers Metasploitable
ping -c 4 192.168.56.30

# Vers Windows 7 (le pare-feu Windows peut bloquer l'ICMP par defaut)
ping -c 4 192.168.56.20
```



Résultats des tests de connectivité depuis Kali : 0% de perte vers les deux cibles.

Observations : les deux cibles répondent (0% de perte). On note que le `TTL` vaut **64** pour Metasploitable (pile Linux) contre **128** pour Windows 7 : cette différence est un indice de *fingerprinting* du système d'exploitation. Ici le pare-feu de Windows 7 laisse passer l'ICMP ; par défaut, il bloque souvent les requêtes d'écho entrantes, auquel cas il faut autoriser la règle « Partage de fichiers et d'imprimantes (Demande d'écho ICMPv4) » ou désactiver temporairement le pare-feu.

3. Tâche 1 : Attaque par déni de service avec Scapy

Les attaques DoS et DDoS sont devenues populaires en raison de leur accessibilité et du faible niveau de compétence nécessaire pour les exécuter. Elles peuvent rapidement consommer les ressources disponibles d'un système ou d'un réseau, les rendant inutilisables.

3.1 Rappels théoriques

Impacts possibles : perte de réputation, désactivation des réseaux, pertes financières, dysfonctionnement des organisations.

Catégories de vecteurs DoS/DDoS :

— **Attaques volumétriques** (saturent la bande passante) : UDP flood, ICMP flood, Ping of Death, Smurf.

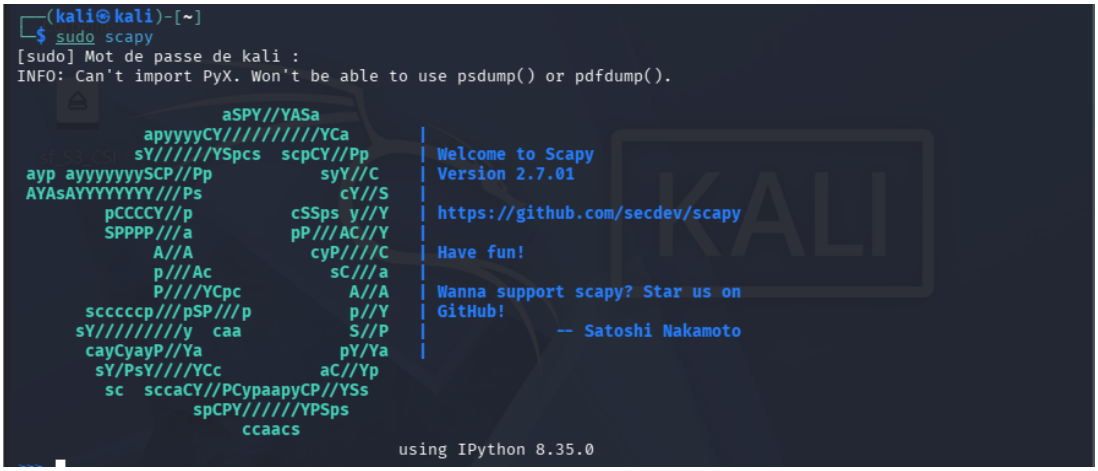
— **Attaques par protocole** (saturent les tables d'état : pare-feux, équilibreurs, serveurs) : SYN flood, ACK flood, sessions usurpées.

3.2 Le module Scapy

Scapy est un programme Python interactif de manipulation de paquets. Il permet de forger, décoder, envoyer, capturer des paquets de nombreux protocoles. Il peut remplacer hping, arpspoof, arping, p0f et certaines parties de Nmap, tcpdump et tshark.

Lancer l'outil sur Kali :

```
sudo scapy
```



Bannière de démarrage de Scapy.

3.3 Manipulations

1. Créer un paquet IP et afficher ses champs avec `display()` :

```
i = IP()
i.display()
```

2. Paramétrer la destination (ici la cible Metasploitable, ou le broadcast) :

```
i.dst = "192.168.56.30"
i.display()
```



Paquet IP avec destination définie (src renseignée automatiquement).

3. Créer un paquet ICMP (echo-request / ping) :

```
ping = ICMP()
ping.display()
```

4. Construire la requête et l'envoyer avec `send()` :

```
requete = (i/ping)
send(requete)

# Envoi massif vers le broadcast du reseau
send(IP(dst="192.168.56.255")/ICMP(), count=100, verbose=1)
```



Envoi de 100 paquets ICMP (`Sent 100 packets`).

3.4 Questions de la tâche 1

Q1. Pourquoi l'utilisation de Scapy n'entraîne-t-elle pas de déconnexion ou d'indisponibilité de la cible, même en envoyant des milliers de paquets ?

Réponse : Scapy fonctionne en espace utilisateur et en Python : il forge et envoie les paquets un par un via la pile réseau, avec un coût d'interprétation élevé par paquet. Le débit obtenu (quelques centaines à quelques milliers de paquets/seconde au mieux) est très loin de ce qu'il faut pour saturer la cible ou le lien. Un outil comme `hping3 --flood` ou un module Metasploit injecte les paquets au plus près du noyau et atteint des débits bien supérieurs. Scapy est donc un outil de *forge et d'analyse*, pas un outil de *saturation*.

Q2. Dans le cadre d'une attaque DoS, quel est l'intérêt d'utiliser Scapy ? Qu'apporte-t-il par rapport à d'autres outils ?

Réponse : Scapy n'est pas fait pour le volume mais pour la *précision* : il permet de fabriquer des paquets malformés ou très spécifiques (champs IP/TCP arbitraires, fragmentation, flags illégaux, usurpation d'adresse source, Ping of Death, etc.) afin de tester la robustesse d'une pile réseau ou de déclencher une vulnérabilité logique (« attaque applicative/par exploitation ») plutôt qu'une simple saturation volumétrique. C'est un outil de prototypage d'attaque et de recherche.

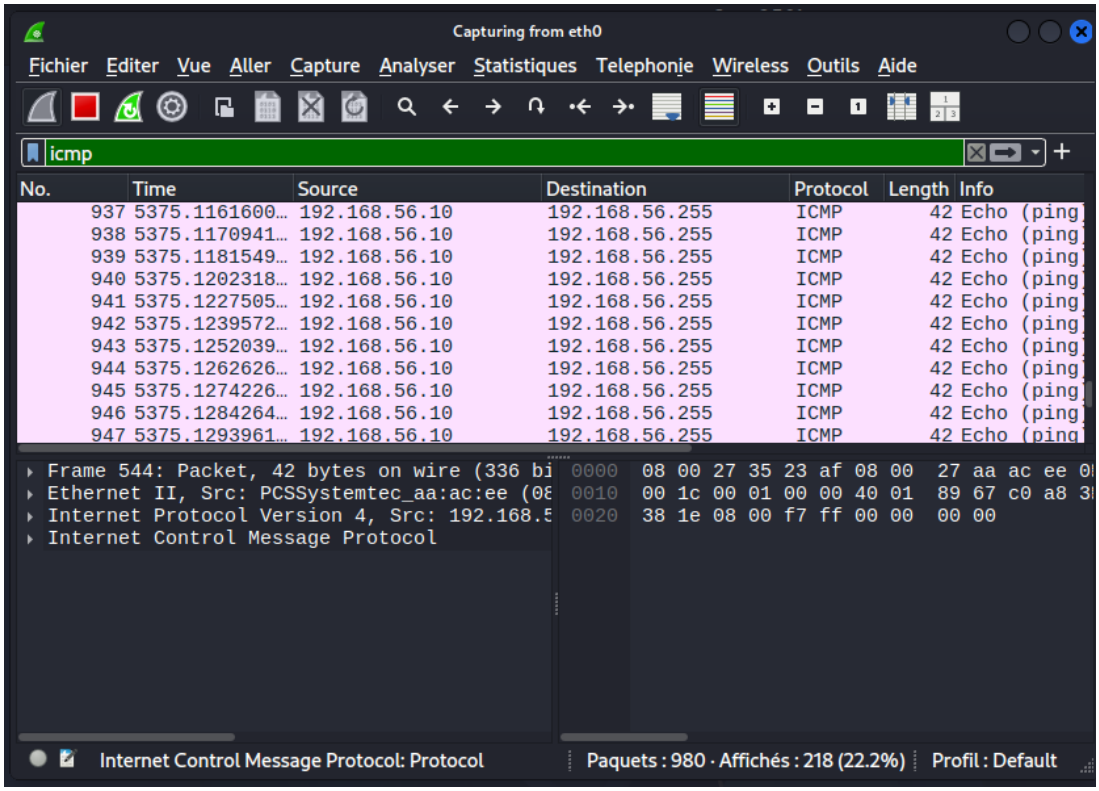
4. Tâche 2 : Analyse du trafic avec Wireshark

Pendant l'envoi des paquets de la tâche 1, on capture et on analyse le trafic. Lancer Wireshark sur Kali et choisir l'interface `eth0` :

```
sudo wireshark &
```

Pour repérer facilement les requêtes ping, filtrer sur le protocole ICMP :

```
icmp
```



Capture Wireshark filtrée sur ICMP : flood de requêtes écho vers le broadcast.

Interprétation : on observe un flux continu de paquets « Echo (ping) request » émis par Kali (`192.168.56.10`, MAC `08:00:27:aa:ac:ee`) vers l'adresse de broadcast `192.168.56.255`. On ne voit *que* des requêtes, sans réponse : par sécurité, les hôtes ne répondent pas aux requêtes ICMP destinées à l'adresse de diffusion (protection contre l'amplification de type Smurf). L'adresse source correspond à l'attaquant (sauf usurpation), ce qui montre qu'un DoS direct non usurpé est facile à tracer.

5. Tâche 3 : Attaque DoS par SYN Flood

L'attaque par inondation SYN exploite le mécanisme d'établissement de connexion TCP en trois temps (**TCP three-way handshake**). L'attaquant envoie un grand nombre de paquets SYN sans jamais terminer la poignée de main : la cible conserve de nombreuses connexions *à moitié ouvertes* (état `SYN_RECV`), ce qui épuise sa table de connexions.

5.1 3.1 SYN Flood avec Metasploit

Le module `auxiliary/dos/tcp/synflood` forge des paquets SYN bas niveau : il requiert donc les privilèges root (capability `CAP_NET_RAW`). On lance donc Metasploit avec `sudo`.

```
# 1. Lancer Metasploit en root (necessaire pour forger les paquets)
sudo msfconsole

# 2. Charger le module
use auxiliary/dos/tcp/synflood

# 3. Voir les options
show options

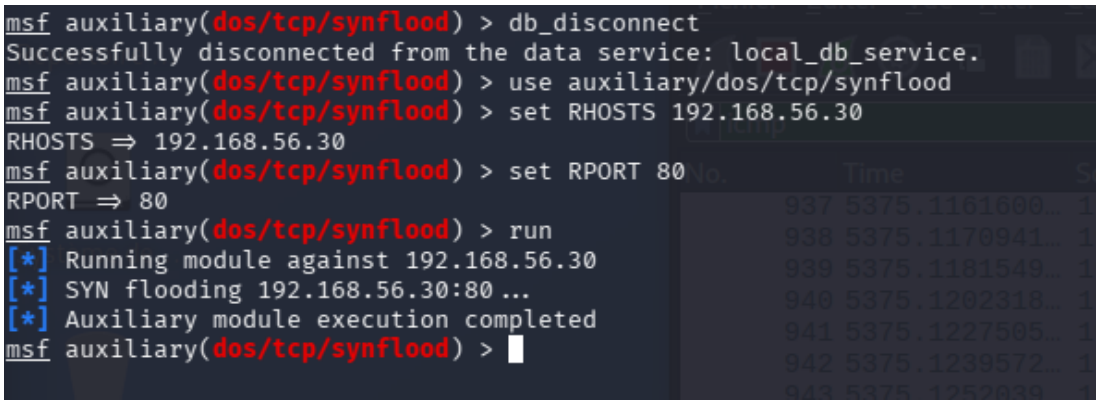
# 4. Configurer la cible (Metasploitable, port 80 ouvert)
set RHOSTS 192.168.56.30
set RPORT 80
```

```
# 5. Lancer l'attaque
run
```

Deux problèmes rencontrés et résolus :

1. `Attempt to create packet socket failed - CAP_NET_RAW may be required` : Metasploit avait été lancé en utilisateur normal, sans droit de forger des paquets. *Solution :* relancer avec `sudo msfconsole`.
2. `Auxiliary failed: Timeout::ExitException` (pile d'appels PostgreSQL) : en root, la base de données de Metasploit (rattachée au compte `kali`) n'est pas joignable et provoque un timeout. *Solution :* détacher la base avec `db_disconnect` avant de lancer le module (le module DoS n'en a pas besoin).

```
db_disconnect
use auxiliary/dos/tcp/synflood
set RHOSTS 192.168.56.30
set RPORT 80
run
```



Metasploit : SYN flood lancé avec succès sur 192.168.56.30 :80 (`SYN flooding 192.168.56.30:80 ...`).

5.2 3.2 SYN Flood avec hping3

```
sudo hping3 192.168.56.30 -S --flood -p 80
```

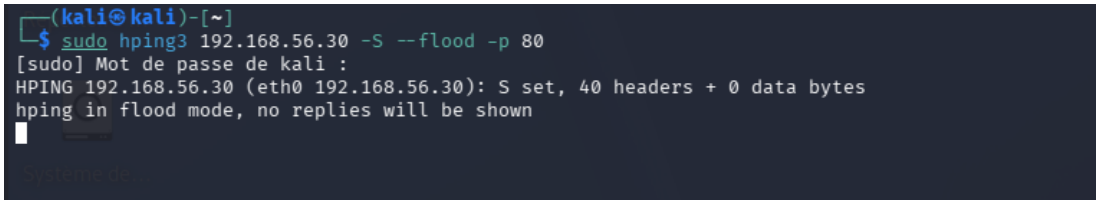
Explication de la commande :

— `192.168.56.30` : adresse IP de la cible (Metasploitable).

— `-S` : positionne le flag **SYN** dans les paquets TCP.

— `--flood` : envoie les paquets le plus vite possible, sans attendre de réponse.

— `-p 80` : port cible (serveur web de Metasploitable).



Lancement du SYN flood avec hping3 en mode `--flood`.

Incident rencontré : un `--flood` à plein débit a fait *tomber l'interface eth0 de la machine attaquante elle-même* (perte de l'IP statique, hping basculant sur `lo` puis « Network is unreachable »).

En cause : NetworkManager tentait un DHCP sur un réseau host-only sans serveur DHCP et flushait l'adresse statique. *Solution :* arrêter NetworkManager, fixer l'adresse manuellement, puis utiliser un débit contrôlé (`-i u200`).

```
sudo systemctl stop NetworkManager
sudo ip addr flush dev eth0
sudo ip addr add 192.168.56.10/24 dev eth0
sudo ip link set eth0 up
```

5.3 3.3 Démonstration de l'impact (et des défenses)

L'objectif est de saturer la file des connexions semi-ouvertes (`SYN_RECV`) de la cible. La démonstration se fait par étapes, ce qui révèle au passage **trois mécanismes de défense** de la pile TCP/IP.

État de référence (avant attaque)

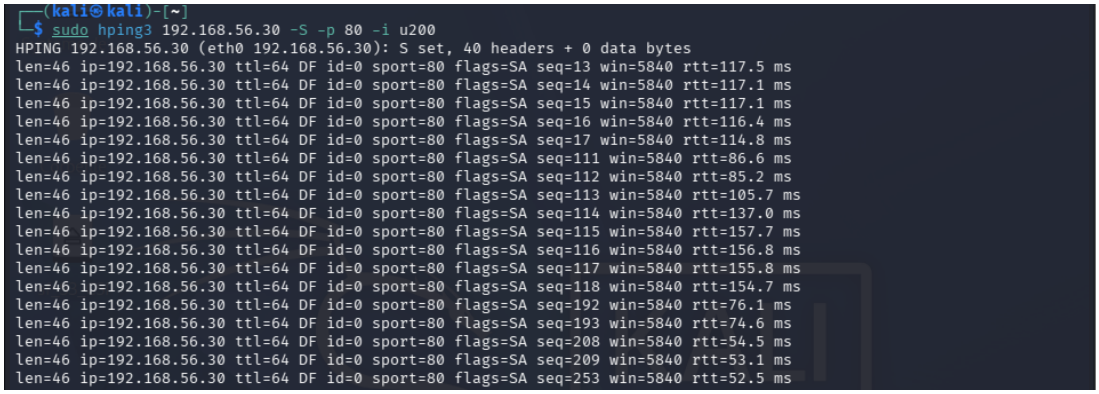
Le serveur web répond normalement (`real 0,22s`) :



Temps de réponse du serveur web avant l'attaque.

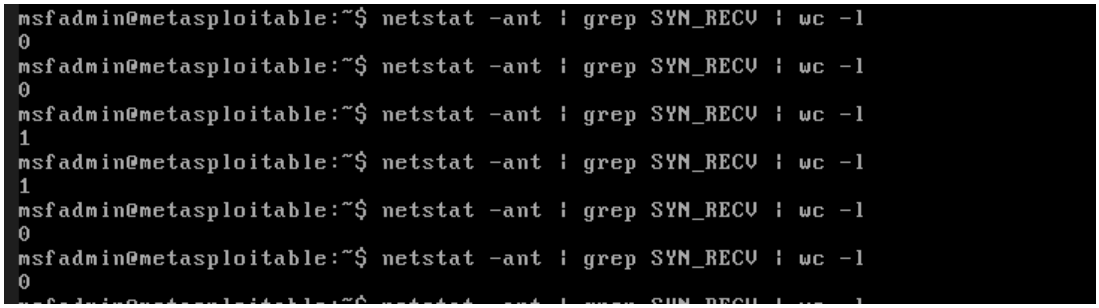
Tentative 1 : flood non usurpé

Après avoir désactivé les SYN cookies sur la cible (`sudo sysctl -w net.ipv4.tcp_syncookies=0`), on lance un flood dont la source est la vraie IP de Kali. La cible répond bien par des SYN-ACK (`flags=SA`, port 80 ouvert)...



La cible renvoie des SYN-ACK vers l'attaquant (`flags=SA`).

... mais le compteur `SYN_RECV` reste à 0 :

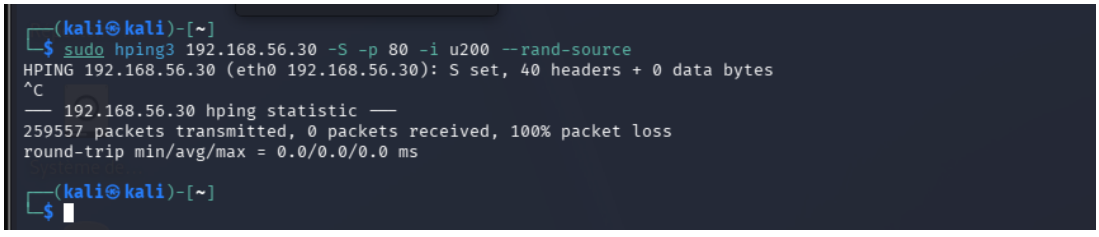


Flood non usurpé : la file `SYN_RECV` ne se remplit pas.

Défense 1 (RST du noyau) : la source n'étant pas usurpée, les SYN-ACK reviennent vers Kali, dont le noyau, qui ne connaît pas ces connexions, répond par un `RST` qui ferme immédiatement les sessions côté cible. La file ne se remplit donc jamais.

Tentative 2 : usurpation aléatoire (--rand-source)

```
sudo hping3 192.168.56.30 -S -p 80 -i u200 --rand-source
```



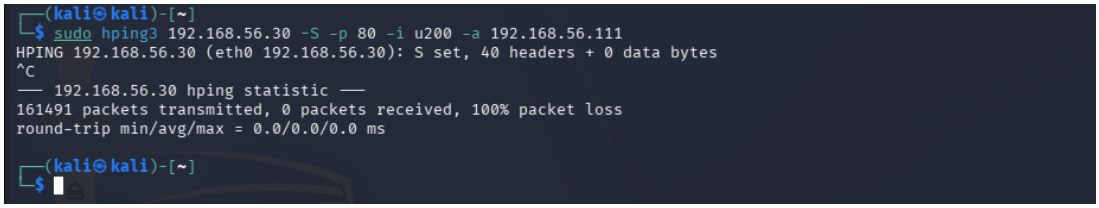
Flood avec sources aléatoires : 100% de perte, file toujours vide.

Défense 2 (rp_filter) : `--rand-source` tire des IP dans tout l'espace IPv4, donc hors du sous-réseau. Le réseau host-only n'ayant pas de passerelle, ces adresses ne sont pas routables en retour : le filtrage de chemin inverse (`rp_filter`) les traite comme des *martians* et les rejette avant toute création d'entrée `SYN_RECV`.

Tentative 3 : usurpation dans le sous-réseau (réussie)

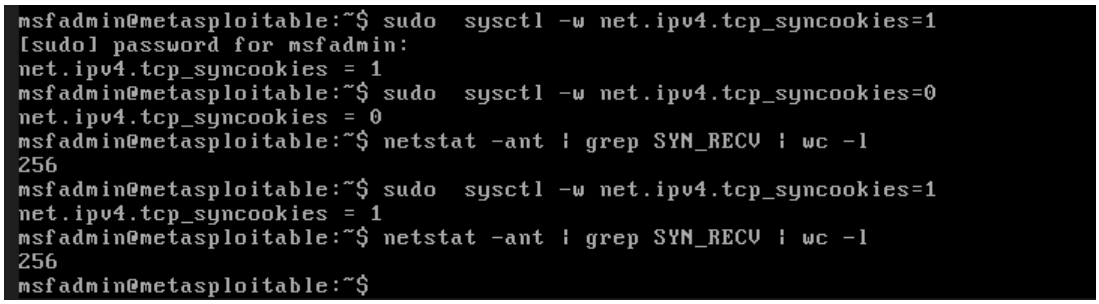
On usurpe une IP *fictive mais appartenant au sous-réseau* (`192.168.56.111`, inexistante) : elle passe le `rp_filter`, et comme l'hôte n'existe pas, aucun RST ne revient.

```
sudo hping3 192.168.56.30 -S -p 80 -i u200 -a 192.168.56.111
```



Flood avec source usurpée dans le sous-réseau.

Résultat : la file `SYN_RECV` se sature à **256** entrées (taille du backlog de la cible) :



File `SYN_RECV` saturée (256) pendant le flood usurpé, puis SYN cookies réactivés.

Défense 3 (SYN cookies) : cette saturation n'a été possible qu'après avoir *désactivé* les SYN cookies. Avec les SYN cookies activés (comportement par défaut de Linux), le serveur n'empile plus les connexions semi-ouvertes : il encode l'état dans le numéro de séquence, et la file reste vide même sous flood. On les réactive impérativement en fin de démonstration :

```
sudo sysctl -w net.ipv4.tcp_syncookies=1
```

5.4 Questions de la tâche 3

Q. Quel est l'impact du SYN flood sur la cible ?

Réponse : lorsqu'il aboutit (défenses désactivées et source usurpée dans le sous-réseau), le flood remplit la table des connexions semi-ouvertes (`SYN_RECV` atteint 256, la limite du backlog). Les nouveaux clients légitimes ne peuvent alors plus établir de connexion : le service devient lent ou indisponible. *Observation importante :* sur une pile correctement configurée (SYN cookies + rp_filter actifs), un SYN flood lancé depuis une seule machine est **neutralisé**, ce qui montre l'efficacité de ces protections.

Comment détecter et se protéger contre les attaques DoS/DDoS ?

— **Détection :** supervision du trafic (pic anormal de SYN, dissymétrie SYN/SYN-ACK), IDS/IPS (Snort, Suricata), NetFlow.

— **Protection :** *SYN cookies* (`net.ipv4.tcp_syncookies`), filtrage anti-usurpation (`rp_filter`, BCP38/uRPF), limitation de débit (rate limiting), pare-feu avec seuils par source, et pour le volumétrique, un service anti-DDoS en amont (scrubbing / CDN).

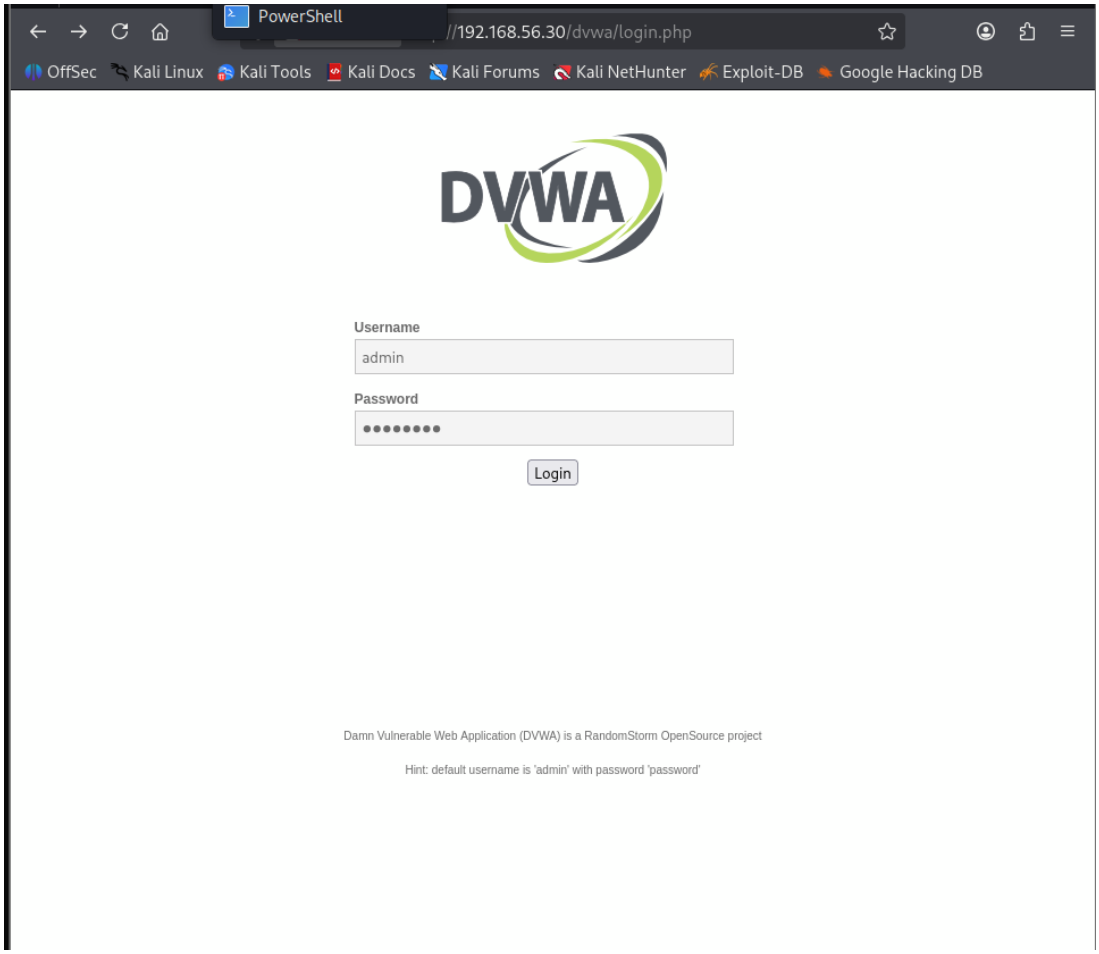
6. Tâche 4 : Sniffing avec Wireshark (identifiants DVWA)

Le sniffing consiste à capturer le trafic réseau à l'aide d'un analyseur de paquets. Lorsque les données ne sont pas chiffrées (HTTP), leur contenu est lisible en clair.

On utilise l'application vulnérable **DVWA** hébergée par Metasploitable 2. Depuis un navigateur, se rendre sur :

```
http://192.168.56.30/dvwa/
```

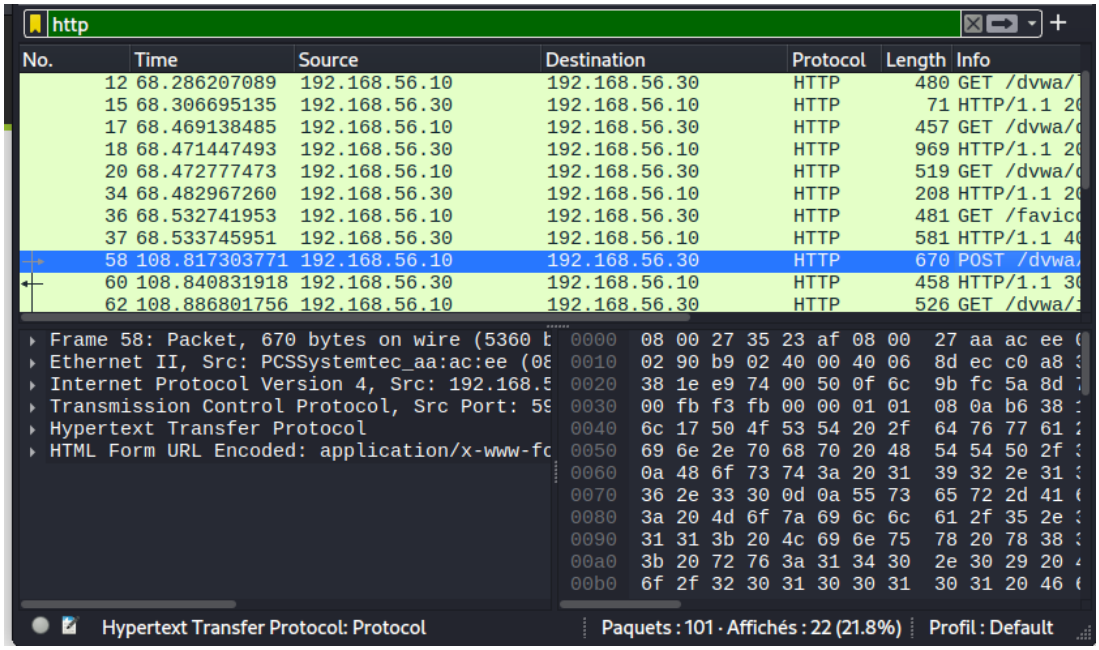
Identifiants par défaut : **Login :** `admin` **Pass :** `password`.



Page de connexion DVWA.

Lancer la capture Wireshark sur `eth0`, filtrer le trafic HTTP, puis se connecter à DVWA :

```
http
```



Trafic HTTP capturé : on repère la requête `POST /dvwa/login.php`.

Repérer la requête `POST /dvwa/login.php` puis « Follow → HTTP Stream » : les champs du formulaire apparaissent en clair.



Flux HTTP suivi : `username=admin&password=password` visibles en clair, suivi d'un `302 Found` confirmant l'authentification réussie.

6.1 Questions de la tâche 4

— **Risques d'un sniffer sur un réseau non sécurisé :** vol d'identifiants, de cookies de session, de données confidentielles transitant en clair.

— **Protection :** chiffrement de bout en bout (HTTPS/TLS, SSH, VPN), segmentation réseau, 802.1X, détection du mode promiscuous.

— **TLS 1.3 :** rend le sniffing *passif* inefficace sur le contenu : la charge utile est chiffrée. Restent exposés : adresses IP, ports, tailles/temps des paquets, et le SNI en clair (sauf ECH).

— **Détecter un sniffing promiscuous :** tests ARP ciblés (réponses d'une carte en mode promiscuous), outils comme `nmap -script sniffer-detect`, analyse des temps de réponse.

— **Indices après incident :** présence de requêtes/réponses ARP anormales (empoisonnement), « duplicate IP address », trafic en clair volumineux vers une machine inattendue.

7. Tâche 5 : Attaque MITM par ARP Spoofing (Ettercap)

Les réseaux locaux commutés empêchent le sniffing passif entre machines. L'usurpation ARP (**ARP spoofing / poisoning**) permet de contourner cela : l'attaquant envoie de faux messages ARP pour que le trafic destiné à une IP soit redirigé vers sa machine. Il se place ainsi « au milieu » des échanges (Man-in-the-Middle), puis relaie le trafic vers la véritable destination.

7.1 Le protocole ARP

L'Address Resolution Protocol associe une adresse IP (couche 3) à une adresse MAC (couche 2). Une machine qui veut communiquer avec une IP locale diffuse une requête ARP « qui a cette IP ? » ; la machine concernée répond avec sa MAC. ARP ne vérifie pas l'authenticité des réponses : c'est cette faiblesse qu'exploite l'empoisonnement.

7.2 Scénario

On intercepte le trafic entre **Windows 7** (192.168.56.20) et **Metasploitable** (192.168.56.30, serveur DVWA), depuis **Kali** (192.168.56.10).

7.3 Mise en œuvre

1. **État initial : table ARP saine de la victime** (sur Windows 7, `arp -a`) :

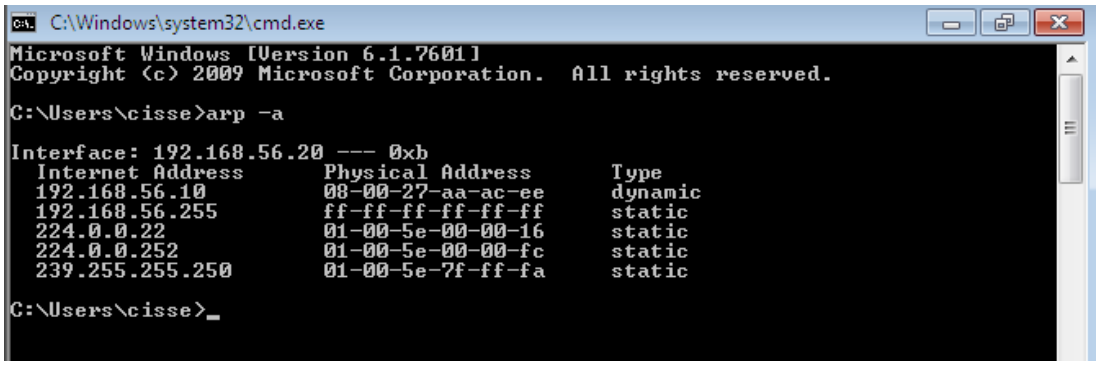


Table ARP de Windows 7 avant l'attaque.

2. **Activer le routage IP sur Kali** (pour relayer le trafic et ne pas couper la victime, faute de quoi le MITM se transforme en déni de service) :

```
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
```

3. **Lancer Ettercap pour empoisonner les deux cibles :**

```
sudo ettercap -T -i eth0 -M arp /192.168.56.20// /192.168.56.30//
```

Ettercap identifie les deux victimes avec leurs *vraies* adresses MAC : `GROUP 1 = 192.168.56.20 (08:00:27:d5:11:03)` pour Windows 7 et `GROUP 2 = 192.168.56.30 (08:00:27:35:23:af)` pour Metasploitable.



Ettercap : ARP poisoning des deux cibles (GROUP 1 / GROUP 2), sniffing unifié démarré.

4. **Vérifier l'empoisonnement côté victime** (sur Windows 7, `arp -a`) :

```
C:\Users\cisse>arp -a

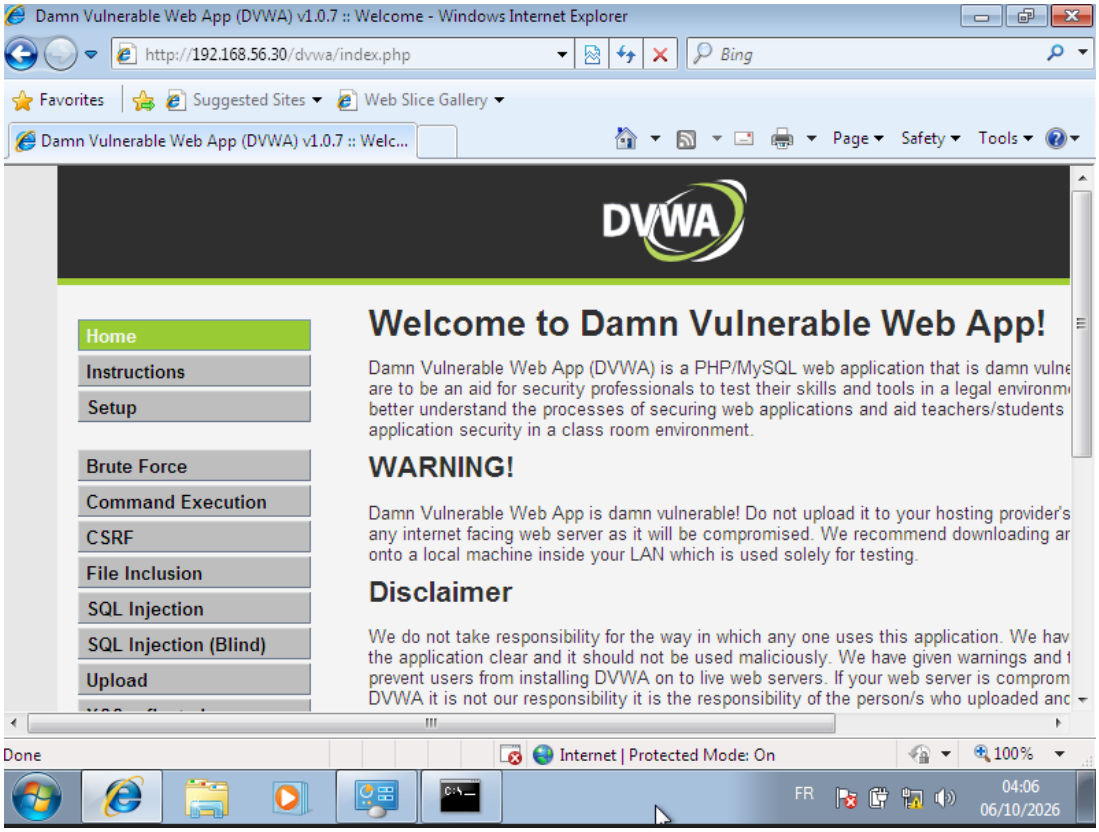
Interface: 192.168.56.20 --- 0xb
  Internet Address      Physical Address      Type
  192.168.56.10         08-00-27-aa-ac-ee     dynamic
  192.168.56.30         08-00-27-aa-ac-ee     dynamic
  192.168.56.255        ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static

C:\Users\cisse>_
```

Table ARP de Windows 7 empoisonnée.

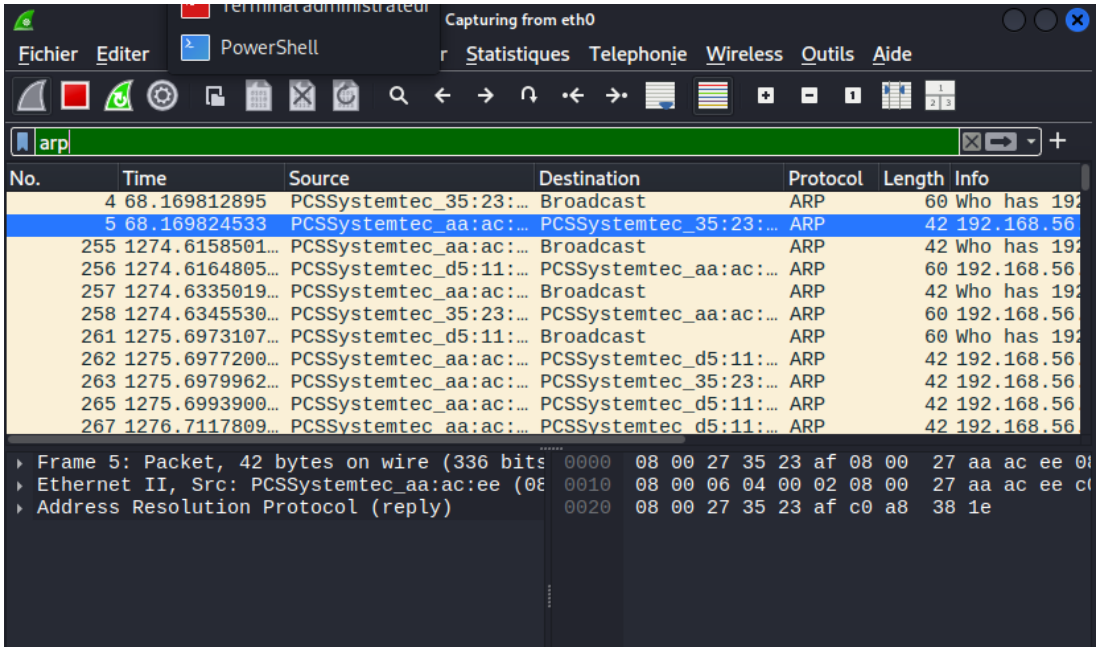
Preuve du MITM : après l'attaque, l'entrée `192.168.56.30` affiche la MAC de Kali (`08-00-27-aa-ac-ee`) au lieu de celle de Metasploitable (`08-00-27-35-23-af`). Les adresses `192.168.56.10` et `192.168.56.30` pointent désormais vers la *même* MAC (celle de l'attaquant) : c'est la signature caractéristique d'un empoisonnement ARP. Le trafic de la victime vers la cible transite donc par Kali.

5. Le trafic de la victime transite par l'attaquant : grâce au routage IP, Windows 7 accède normalement à DVWA, sans interruption perceptible (le MITM reste furtif).



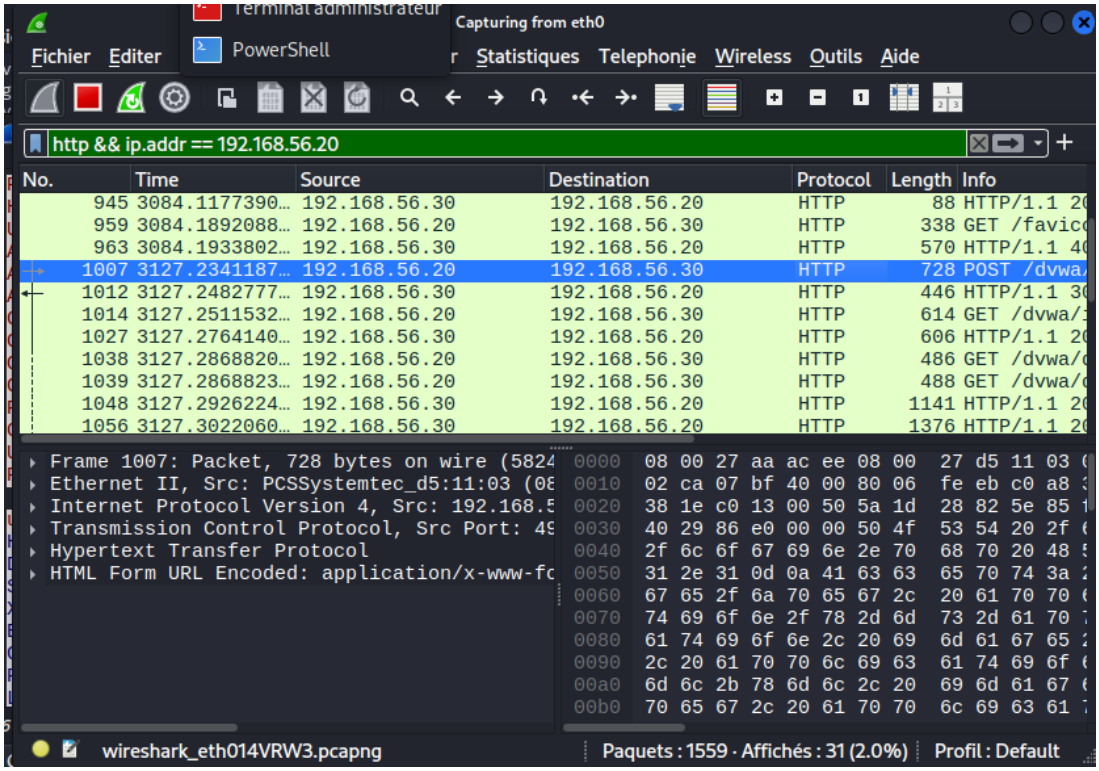
Windows 7 accède à DVWA à travers le MITM (routage transparent).

6. Observer les paquets d'empoisonnement avec Wireshark (filtre `arp`) : on voit les réponses ARP falsifiées émises en continu par Kali (`PCSSystemtec_aa:ac:ee`).



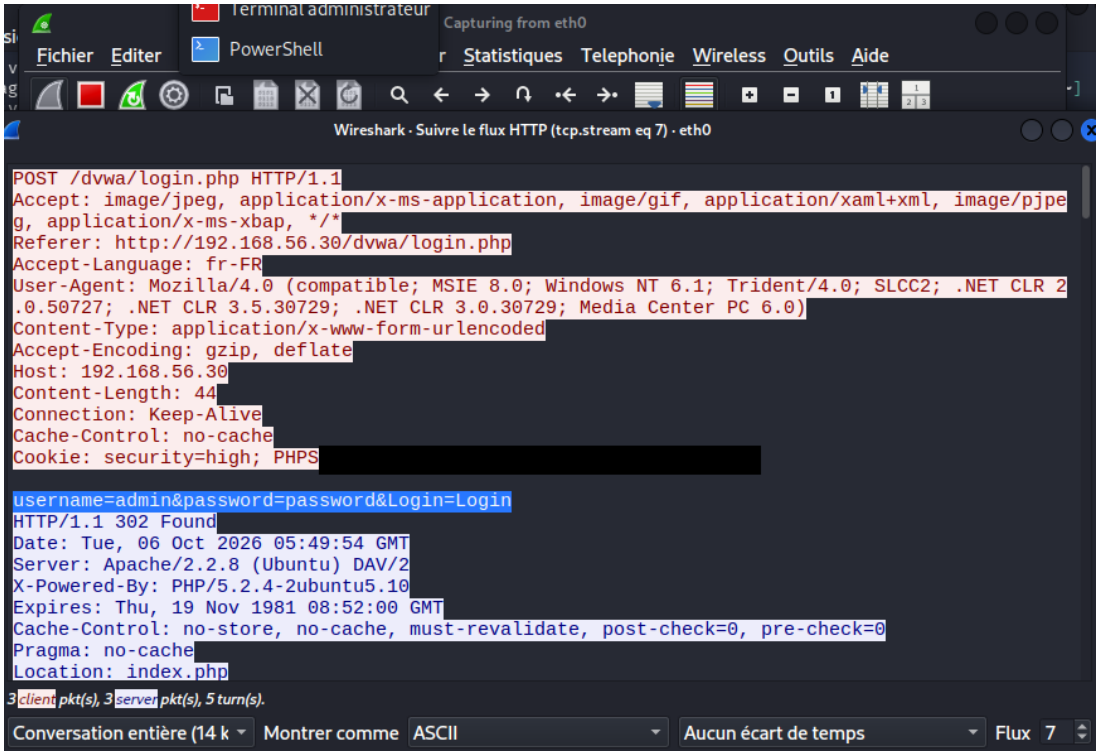
Wireshark : réponses ARP falsifiées émises par l'attaquant.

7. Intercepter le trafic HTTP de la victime (filtre `http && ip.addr == 192.168.56.20`) : lorsque Windows 7 se connecte à DVWA, la requête `POST /dvwa/login.php` transite par Kali. On la repère avec pour *source* `192.168.56.20` et pour MAC Ethernet `08:00:27:d5:11:03` (celle de Windows 7).



Wireshark sur Kali : le POST de la victime (Src = 192.168.56.20) est bien intercepté via le MITM.

En suivant le flux HTTP, les identifiants réapparaissent en clair. Le `User-Agent` (`MSIE 8.0; Windows NT 6.1`) confirme que la requête provient bien de **Windows 7**, et non de l'attaquant, ce qui prouve l'interception du trafic d'un tiers.



Flux HTTP de la victime : `username=admin&password=password` en clair, User-Agent Windows 7.

7.4 Questions de la tâche 5

— **Qu'est-ce que l'ARP spoofing et comment permet-il un MITM ?** L'attaquant envoie de fausses réponses ARP associant son propre MAC aux IP des deux victimes ; leur trafic lui est alors acheminé, puis relayé : il voit et peut modifier les échanges.

— **Se protéger :** entrées ARP statiques pour les hôtes critiques, *Dynamic ARP Inspection* (DAI) + DHCP snooping sur les switches, segmentation VLAN, chiffrement (HTTPS/VPN) pour limiter l'impact, outils de détection (arpwatch).

— **Techniques combinables :** DNS spoofing, DHCP starvation/rogue DHCP, SSL stripping, pour une attaque plus furtive et plus large.

8. Réflexion globale et conclusion

1. **Difficultés rencontrées et solutions :**
   — *Metasploit en utilisateur normal* : erreur `CAP_NET_RAW` à la création du socket de paquets → relance en `sudo`.
   — *Base PostgreSQL inaccessible en root* : `Timeout::ExitException` → `db_disconnect` (le module DoS n'a pas besoin de la base).
   — *Chute de l'interface eth0 de Kali* pendant un `--flood` : NetworkManager tentait un DHCP sur un réseau host-only sans serveur DHCP et flushait l'IP statique → arrêt de NetworkManager + adresse fixée manuellement + débit contrôlé.
   — *SYN flood sans effet* malgré des milliers de paquets : neutralisé successivement par le RST du noyau (source non usurpée), le `rp_filter` (source aléatoire hors sous-réseau) puis les SYN cookies → saturation obtenue seulement avec une source usurpée *dans* le sous-réseau et les SYN cookies désactivés.
2. **Application en milieu professionnel :** ces techniques sont la base des tests d'intrusion internes (audit de segmentation, résistance au DoS, exposition des protocoles en clair) et nourrissent directement le volet défensif (durcissement, détection, supervision).

3. **Mesures de défense recommandées :**
   — *DoS/DDoS :* SYN cookies, rate limiting, anti-usurpation (BCP38), anti-DDoS amont, supervision.
   — *Sniffing :* chiffrement systématique (TLS, VPN), segmentation.
   — *MITM/ARP :* DAI + DHCP snooping, ARP statiques, VLAN, arpwatch.
4. **Hiérarchie des contre-mesures :** prioriser d'abord ce qui réduit la surface et l'impact systémique : (1) durcissement et correctifs (éviter l'exploitation), (2) chiffrement de bout en bout (neutralise le sniffing et une grande partie du MITM), (3) protections réseau (DAI, anti-DDoS), car elles couvrent le plus grand nombre de scénarios pour un effort maîtrisé.
5. **Automatiser la détection/réponse à grande échelle (type EternalBlue, parc de 10 000 machines) :** inventaire et gestion centralisée des correctifs (ex. WSUS/SCCM), EDR déployé sur chaque poste avec règles de détection SMB/MS17-010, SIEM pour la corrélation, et playbooks SOAR pour isoler automatiquement une machine compromise du réseau.

Conclusion. Ce laboratoire a permis de mettre en pratique la chaîne offensive DoS → sniffing → MITM sur un environnement isolé, et surtout d'en déduire les contre-mesures. La leçon principale est que la plupart de ces attaques exploitent soit l'absence de chiffrement, soit la confiance implicite de protocoles anciens (ARP, TCP) : le durcissement, le chiffrement de bout en bout et la supervision en sont les parades essentielles.

9. Annexe : commandes utilisées, rôles et options

Cette annexe regroupe l'ensemble des commandes employées durant le lab, avec leur rôle et le détail de leurs principales options.

Commande	Rôle et options
Réseau et diagnostic	
`ip a`	Affiche les interfaces réseau avec leurs adresses IP et MAC (Kali).
`ifconfig`	Équivalent historique de `ip a` (utilisé sur Metasploitable).
`ipconfig`	Affiche la configuration IP sous Windows.
`ping -c 4 <ip>`	Teste la connectivité via ICMP. `-c 4` limite l'envoi à 4 requêtes.
`arp -a`	Affiche la table ARP (correspondances IP vers MAC) ; sert à prouver l'empoisonnement côté Windows.
Remise en état du réseau Kali	
`sudo systemctl stop NetworkManager`	Arrête NetworkManager pour qu'il cesse de flusher l'IP statique (réseau host-only sans DHCP).
`sudo ip addr flush dev eth0`	Supprime toutes les adresses de l'interface eth0.
`sudo ip addr add 192.168.56.10/24 dev eth0`	Fixe l'adresse statique ; le suffixe `/24` crée automatiquement la route vers le sous-réseau.
`sudo ip link set eth0 up`	Active (monte) l'interface eth0.
Tâche 1 : Scapy	

suite page suivante

Commande	Rôle et options
`sudo scapy`	Lance l'interpréteur Scapy (root requis pour forger des paquets).
`i = IP()`	Crée un paquet IP. Attributs utiles : `.dst` (destination), `.src` (source).
`ping = ICMP()`	Crée un paquet ICMP de type echo-request.
`i.display()`	Affiche en détail les champs d'un paquet.
`requete = (i/ping)`	Empile les couches : IP encapsule ICMP.
`send(pkt, count=100, verbose=1)`	Envoie le paquet. `count` fixe le nombre d'envois, `verbose` l'affichage.
Tâches 2, 4 et 5 : Wireshark (filtres d'affichage)	
`sudo wireshark`	Lance l'analyseur de trafic sur l'interface choisie (eth0).
`icmp`	Filtre d'affichage ne gardant que les paquets ICMP.
`http`	Filtre d'affichage ne gardant que le trafic HTTP.
`arp`	Filtre d'affichage ne gardant que les paquets ARP.
`http && ip.addr == 192.168.56.20`	Combine deux conditions : HTTP impliquant l'adresse de la victime.
Follow HTTP Stream	Reconstitue l'intégralité d'un échange HTTP (requête + réponse) en clair.
Tâche 3 : SYN flood et mesure d'impact	
`sudo msfconsole`	Lance la console Metasploit (sudo nécessaire pour la capability `CAP_NET_RAW`).
`use auxiliary/dos/tcp/synflood`	Charge le module d'attaque SYN flood.
`show options`	Liste les paramètres du module (requis ou optionnels).
`set RHOSTS <ip>` / `set RPORT 80`	Définit l'adresse de la cible et le port visé.
`db_disconnect`	Détache la base PostgreSQL (évite le timeout quand msfconsole tourne en root).
`run`	Lance l'exécution du module.
`sudo hping3 <ip> -S --flood -p 80`	Inonde la cible de paquets TCP SYN. Options : `-S` active le flag SYN ; `--flood` envoie au débit maximal sans afficher les réponses ; `-p 80` vise le port 80 ; `-i u200` limite le débit (1 paquet toutes les 200 microsecondes) ; `-a <ip>` usurpe une IP source fixe ; `--rand-source` utilise des IP sources aléatoires.
`cat /proc/sys/net/ipv4/tcp_syncookies`	Affiche l'état des SYN cookies (1 = activés, 0 = désactivés).
`sudo sysctl -w net.ipv4.tcp_syncookies=0`	Modifie un paramètre noyau à chaud ; ici désactive (0) ou réactive (1) les SYN cookies.

suite page suivante

Commande	Rôle et options
`netstat -ant | grep SYN_RECV | wc -l`	Compte les connexions semi-ouvertes. `-a` toutes les connexions, `-n` format numérique, `-t` TCP uniquement ; `grep` filtre l'état `SYN_RECV`, `wc -l` compte les lignes.
`time curl -m 5 http://<ip>/`	Mesure le temps de réponse du serveur web. `-m 5` fixe un délai maximal de 5 s, `time` chronomètre l'exécution.
Tâche 5 : MITM par ARP spoofing	
`echo 1 | sudo tee /proc/sys/net /ipv4/ip_forward`	Active le routage IP sur Kali pour relayer le trafic de la victime (sinon le MITM coupe la communication).
`sudo ettercap -T -i eth0 -M arp /192.168.56.20// /192.168.56.30//`	Lance l'empoisonnement ARP entre deux cibles. `-T` interface texte, `-i eth0` interface d'écoute, `-M arp` méthode (ARP poisoning), `/ip//` désigne chaque cible.